CA4022 – Data (Analytics) at Speed and Scale

Alessandra Mileo

alessandra.mileo@dcu.ie

Session topics

1. Anonymous classes (brief revision from Java 5/6/7)
2. Lambda expressions
3. Java Collections (revision)
4. Java 8 Streams

NOTE: You will need these concepts later in the module in order to work with Apache Spark

Topic 1: Anonymous Classes

Functional Programming (FP) and Big Data

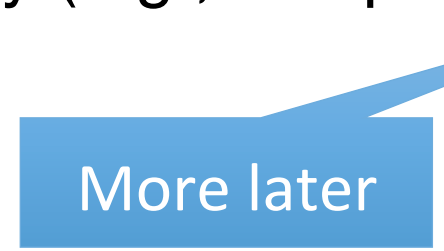
- "Big data" frameworks like Spark and "Big data" programming models like MapReduce are heavily geared to the FP paradigm
- Much easier parallelization of task, e.g., because (pure) FP doesn't allow for modification of shared resources
- FP makes it easier to formulate *bulk*, *aggregate* and *pipelined* operations on data collections and streams
- Easier optimization of complex chains of operations due to *laziness*
- Can easily deal with infinite data structures (also facilitated by *laziness*)

NOTE: Java isn't optimal for functional programming (look at, e.g., Scala, Haskell, Swift, Ocaml/F# instead for better support), but improved a lot in this regard with Java 8

Functional Programming and Java 8

Java 8 isn't a real FP language, but you can use the following guidelines to use it in a functional way:

- Avoid mutation (in-place modification of existing data instead of creating a new (modified) object), where possible
- Avoid global variables (e.g., public static fields)
- Avoid side effects of functions: the results of functions/methods should ideally only depend on their arguments, and methods/functions should not manipulate shared state
- Where possible, rely on inherently "parallel" data structures instead of designing parallel solutions manually (e.g., use parallel streams with Java 8)



Anonymous classes

Inner classes without a name, important for creating *function objects*: objects whose purpose it is to store or pass around a function.

1) Suppose you want to call a method with some other function **`fn`** as an *argument* (*), e.g.,

```
x = someMethod(a,b,fn,c);
```

2) We "wrap" **`fn`** inside of a new object `o` whose class `C` has **`fn`** as a method: inside the body of `someMethod`, we can then call `o.fn(…)`

3) Since we need class `C` only for this purpose, it doesn't need a name

These are precursor of Java 8 *Lambda expressions*

(*) Note: we are talking about using the *entire* function fn as argument, not about using the result of calling fn as argument. In Java < 8, functions aren't *first-class citizens*, so this is not directly possible there.

Anonymous classes: example

Creates a new object of an anonymous class which implements interface `ActionListener`

```
menuItem.addActionListener(new ActionListener() {
   public void actionPerformed(ActionEvent event) {
   // the function "wrapped" inside the object
            …Code of the function…
 }
});
```

- A function (like `addActionListener`) which takes another function (like `actionPerformed`) as an argument or returns a function as result is called *higher-order function*
- The Apache Spark programming model makes heavy use of higher-order functions

Anonymous classes: example

- Purpose: we want to pass function `actionPerformed` into method `addActionListener` as an argument
- Since this is not directly possible in Java pre-8, a new object (the *function object* which "wraps" `actionPerformed`) is created and used as argument for method `addActionListener`
- The new object is an instance of an unnamed (i.e., anonymous) class which implements interface `ActionListener`
- The interface needs to declare the function which we want to pass into `addActionListener` as an argument, that is, function `actionPerformed`
- The only purpose of this anonymous class is to provide a method implementation (of method `actionPerformed(...)` which is specified in interface `ActionListener`) and to pass this method on to some other method (here: `addActionListener(...)`).
- The example is from GUI programming (Java Swing), but it is a universal pattern which is seen in many other use cases too

Anonymous classes: example

This is how the interface looks like:

```
public interface ActionListener extends EventListener {
     public void actionPerformed(ActionEvent e);
     // abstract method
 }
```

When you create the function object using `new ActionListener() {…}`, you provide an implementation of abstract method `actionPerformed`.

- In Java 8, such an interface is called a *functional interface* (guess why?)
- Later, such interface are also used for determining the type of *lambda expressions*

Anonymous classes

- You could likewise use an ordinary class (in the example before, you'd create a normal class which implements `ActionListeners` and pass on an object of this class to `menuItem.addActionListener()`)
- But that would require even more coding overhead. And since you don't require this class elsewhere, the class name would be unnecessary.
- Anonymous classes can also be used to specify more than one method. Also, instead of an interface, the anonymous class could also extend a class, using exactly the same syntax pattern:

```
new Superclass() {
    (...overriding/implementing methods of SuperClass...)
  }
```

End of Topic 1: Anonymous Classes

Topic 2: Lambda Expressions

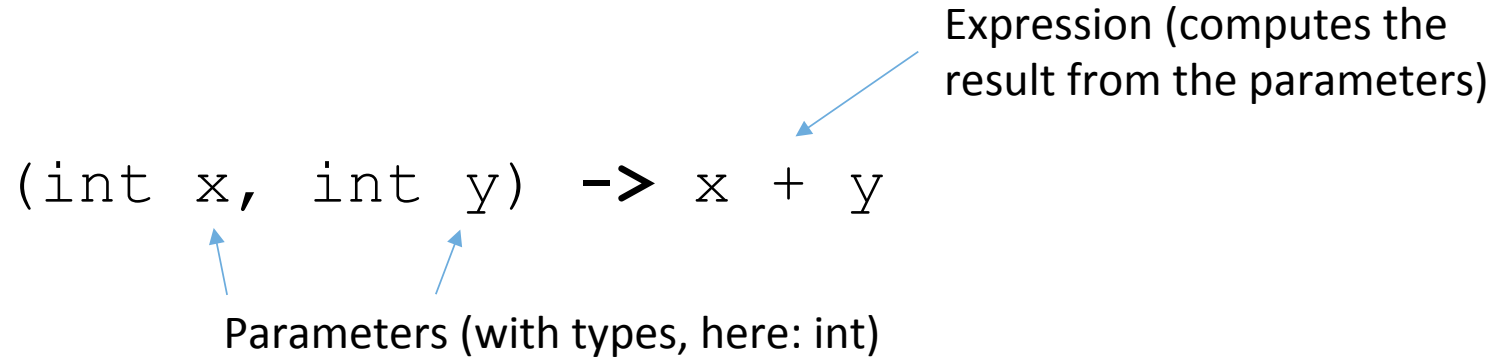
Lambda expressions

- Anonymous classes require knowledge of "traditional" Java only
- However, they have severe shortcomings:
  - Verbose, difficult to read, not very intuitive
  - Runtime overhead: Java creates an object just to represent a function
- In Java 8 we use lambda expressions instead to "pass around functions"
- Lambda expressions are anonymous functions (functions without names)
- *Example*: a lambda expression which represents the *addition* operation:

```
(int x, int y) -> x + y
```

This function takes two integer arguments and gives their sum.

Lambda expressions: a closer look



- Parameters are optional. If there are no parameter, write () before the arrow
- Lambda expressions can use arbitrary blocks of code to produce a result, e.g.,

```
x -> {
        double d = 5.0;
        System.out.println("x: " + x);
        doubld dx = d * x;
        return dc - 7.2;
      }
```

Lambda expressions: a closer look

- Effectively, a lambda expression represents a method as an object
- They can be used almost like any other objects (e.g., they can be assigned to variables and fields, passed on as arguments of methods, or even used as arguments into other lambda expressions…)
- You can provide a lambda expression everywhere where an object with a matching *functional interface* as type is expected
- A functional interface is an interface with a single abstract method (just as the interfaces we used to create anonymous classes)

Lambda expressions: functional interfaces

- Java 8 has a number of pre-defined functional interfaces, e.g., for predicates (Boolean lambda expressions), consumers, and even java.lang.Runnable

```
Predicate<String> pred = (String s) -> { s.equals("Tom") };
```

- You can also create your own functional interfaces using a new annotation:

```
@FunctionalInterface
public interface MyFnInterface {
      public Double doSomething(Double x);
      // the method to which the lambda expression corresponds
}
```

Lambda expressions: functional interfaces

Once we have/know the functional interface, we can use it as the *type* of the lambda expression. E.g., as type of a variable or as type of a parameter of some other method.

```
Double someOtherMethod(MyFnInterface fn) {
      return fn.doSomething(4.3);
      // we evaluate the lambda expression
}
Double r = someOtherMethod((Double x) -> x*2.0);
System.out.println(r);
```

Note: Lambda *abstractions*

- In Computer Science, lambda expressions (the Java 8 term) are more commonly known as *Lambda abstractions* (but these have a somewhat different syntax)
- Lambda abstractions are a corner stone of functional programming
- Their first use was as the constituents of the Lambda calculus
- They are supported by many modern programming languages (including, e.g., Python, JavaScript and C++ 11), not just "real" functional programming languages

Lambda expressions in Big Data Analytics

- Lambda expressions are useful for passing functions as objects
  - E.g., if you want to deploy a task to a remote computer, you can wrap this task in a Lambda expression and submit it to the remote machine.
- They are used with higher-order functions (functions which take functions as arguments), in particular for manipulation, filtering and aggregation of collections (aggregate and other bulk operations).
- Pipelines of such manipulation functions are a crucial concept of modern Big Data Analytics (BDA).
- They generally lead to more compact and readable code

Lambda expressions in Big Data Analytics

- Recall *data parallelism* - a typical use case for lambda expressions in BDA
- Assume you can split your data into several segments
- Next you want to apply the same function `fn` to each of these segments (all applications of `fn` to be performed ***in parallel, independently from each other***)
- You might want to use a software framework for the actual distribution onto different CPUs. You need to tell the framework which function `fn` to apply.
- Pseudocode:

```
foreEachInParallel(segment in data) {
    apply(fn, segment);   //applies function fn to the data segment
}
```

Use a lambda expression to represent fn

Lambda expressions in Big Data Analytics

- A more concrete example: creating objects of task classes ("runnable object") for multithreading…
- You can do this using a named class (see multithreading), but often you want to avoid the overhead of creating a new task class. Anonymous classes could also be used, but still to verbose. So we use lambda expressions…

Lambda expressions in BDA (example)

```
public class RunnableTest {
  public static void main(String[] args) {
     // Anonymous class which implements interface Runnable (old-style Java)
     Runnable r1 = new Runnable() {
        @Override public void run(){
             System.out.println("Hello world one!");
       }
     };
     // Lambda Runnable (Java 8) as a better alternative to the above:
     Runnable r2 = () -> System.out.println("Hello world two!");
     // much more concise and easier to read

  … (we can now take object r1 or r2 and pass it into a thread constructor)
}}
```

Lambda expressions: another example

For full example see http://www.oracle.com/webfolder/technetwork/tutorials/obe/java/Lambda-QuickStart/index.html

```
// Calculate average age of pilots old style (Java 7):

System.out.println("== Calc Old Style ==");
int sum = 0;
int count = 0;

for (Person p:pl){
  if (p.getAge() >= 23 && p.getAge() <= 65){
    sum = sum + p.getAge();
    count++;
  }
}
long average = sum / count;
System.out.println("Total Ages: " + sum);
System.out.println("Average Age: " + average);
```

Lambda expressions

```
// Calculating the sum of persons' ages new style (Java 8):

Predicate<Person> allPilots = p -> p.getAge() >= 23 && p.getAge() <= 65;

long totalAge = pl.stream()
        .filter(allPilots)
        .mapToInt(p -> p.getAge())
        .sum();

// Get average of ages, using a "parallel stream"
OptionalDouble averageAge = pl
        .parallelStream()
        .filter(allPilots)
        .mapToDouble(p -> p.getAge())
        .average();
```

A lambda expression with Boolean result type - ideal for filtering things

Remark: we will see later what stream(), parallelStream(), map(), etc mean. Until then, think of streams as a kind of list.

End of Topic 2: Lambda Expressions

Topic 3: Java Collections

Collections and Java 8 data streams

- Most of you have already used the *Collections* API in Java
- It provides fundamental data structures such as lists, maps and queues, for example `class ArrayList`
- In Java 8, there is a new way to operate on collections: (data) streams(*) and parallel data streams
- Working with streams follows the same pattern that you will later use with Spark (and RDDs)
- They are also another use case for lambda expressions
- Before we turn to the new Java 8 streams, we briefly revise the Collections API…

(*) "Stream" is an overloaded term. Java 8 streams are not the same as the older Java's I/O streams (e.g., for file or console operations), although they have a few things (and the core idea) in common

Collections

- Arrays are only efficient in case we require only random access (but no insertion or deletion) and the length of the collection does not need to change.
- For other use cases, there are Java data structures which are more appropriate, for example:
  - ArrayLists
  - Linked lists
  - Stacks and queues
  - Maps (a.k.a. dictionaries in some other languages)
- These and more are provided by the *Java Collections API a.k.a. Java Collections Framework.*

Collections

- A *collection* in this sense is an instance of one of the *Collections classes*
- The Collections Framework forms a part of the Java API (`java.util`)
- A collection can be seen as a sort of container which stores a number of objects and allows for operations in order to modify and examine the items (elements) within this container
- Similar to arrays, but more powerful and flexible (albeit slower, if no dynamic size of the collection is required)

Collections size and type

- Collections have a *dynamic size*…
- In contrast to plain arrays, they allow to add or delete objects in a way that the size of the collection is different after the operation.
- Collections can contain only objects as elements: in order to store a primitive value (like `int`), this value needs to be "boxed" first, using a *wrapper class*. In most cases, Java does this for you automatically.

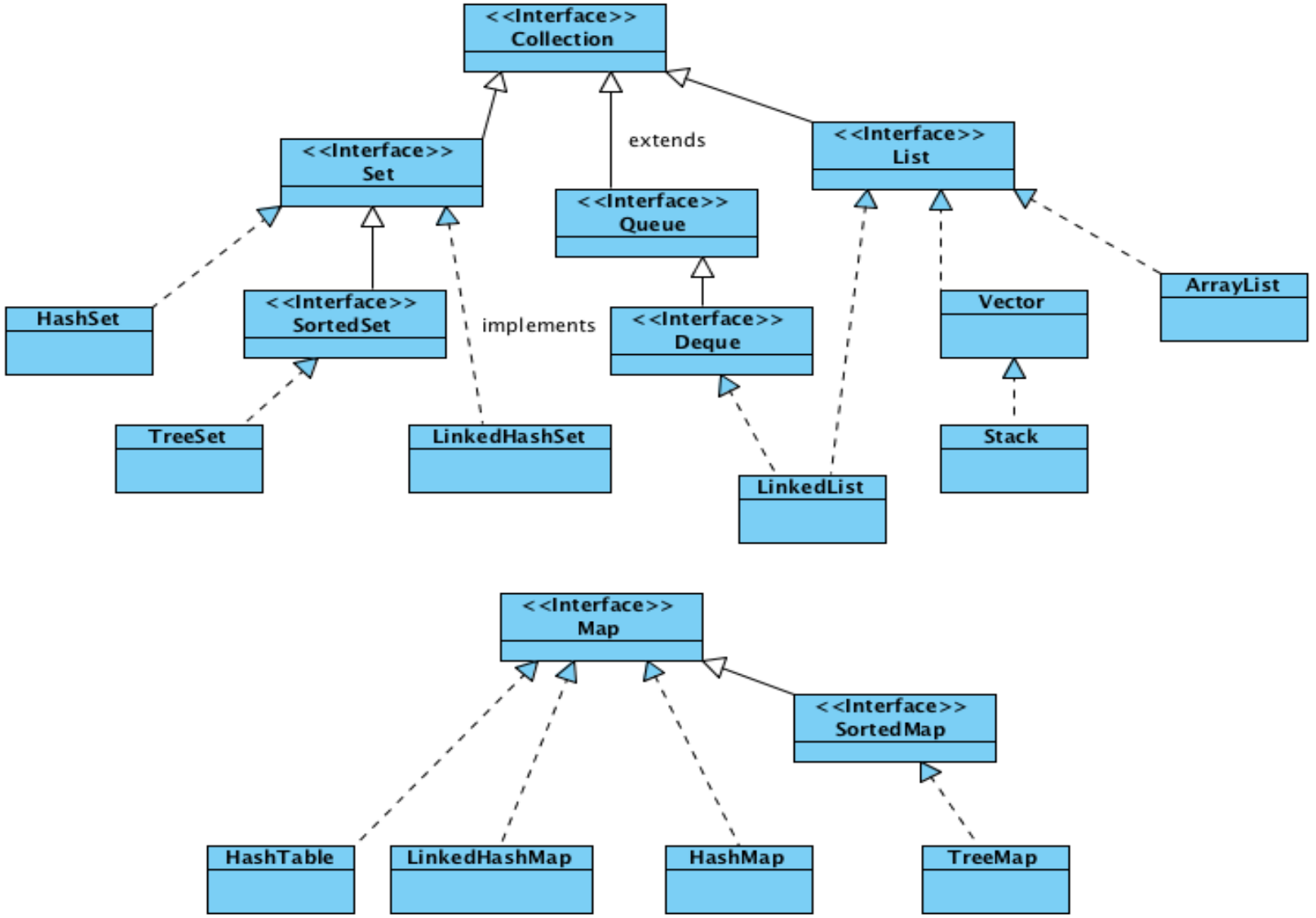
Collections in Big Data Analytics

- In Big Data Analytics (BDA), the concept of collections will translate into large-scale collections of data items, stored and processed in a distributed fashion in a cluster of machines
- Their *use* (from a programmer's point of view) is almost the same in BDA, "only" the processing and the way they are stored changes (e.g., distributed handling on a cluster of computers instead of a single machine)

Collection interface

- Most collection classes and interfaces in Java implement interface `Collection`.
- This interface specifies the basic operations which are allowed for most (but not all) kinds of collections:
  - `add(e)`: adds object `e` to the collection.
  - `remove(e)`: removes one object `e` from the collection.
  - `clear()`: removes all objects from the collection.
  - `contains(e)`: searches the collection. Results in `true` *iff* `e` is contained in the collection.
  - `size()`: results in the number of elements currently in the collection
  - `equals()`: compares the collection content-wise with another collection.
  - `isEmpty()`: `true` iff collection is currently empty.
  - …plus a few more methods

The Collections Framework

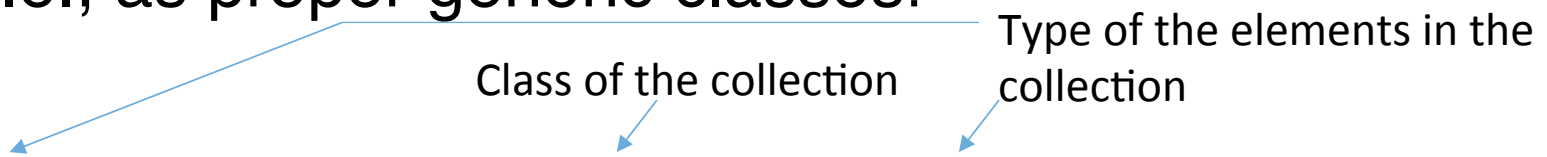


The Collections Framework: types

- Java collections are *generic* classes/interfaces: you can specify the data type (the *type argument*) of the items within a certain collection (e.g., Integer, Double, or String)
- The type argument is a class or interface name within <…> brackets after the basic collection type, e.g., `List<Integer>`
- Always use collections with type arguments, as the compiler couldn't check operations on the collection's items for type correctness otherwise.
- E.g., if you want to store only objects of class X in a collection (without type argument), but by mistake you put an object of class Y into this collection, the compiler couldn't discover this mistake (instead the program would fail at runtime, e.g., if it tries to call an X-method on a Y-object)

The Collections Framework: type

- Therefore, we should use collections only together with a type argument, i.e., as proper generic classes:




```
Set<GeometricShape> s = new HashSet<GeometricShape>();
s.add(new Circle(0.5)); // lets add a few elements…
Circle c = new Circle(123.456);
s.add(c);
s.add(new Rectangle(1.2, 7.8));
s.remove(c); // removes an element
```

- This way, it is not possible anymore to add a "wrong" object:

```
s.add(new Bird()); // fails at compile time!
```

Traversing a collection

- How to traverse the elements of a collection (e.g., a set, a list, a queue, or a map)?
- "Traverse" means here: to visit each of the items in a collection, one after another
- For some collections, we can simply use a plain for-loop, as shown on the next slide...

Traversing a collection: example

```
ArrayList<String> myArrayList = new ArrayList<String>();
myArrayList.add("aaa");
// let's add a few elements
...
myArrayList.add("zzz");
...
// Print all elements:
for(int i = 0; i < myArrayList.size(); i++)
    System.out.println(myArrayList.get(i));
```

Traversing a collection: iterators

- Some collections don't have a `get()` method. But there are other possibilities:
  E.g., to traverse a stack, we could subsequently pop all its elements from the stack (which is empty at the end of this procedure)
- But how to traverse, e.g., an unordered set?
- *Iterators* provide a uniform way to traverse (almost) all kinds of collections....

Traversing a collection: iterators

- Interface `Collection` (which most of the Collection-classes like `ArrayList` implement) provides a method `iterator`, which returns such an *iterator* for the respective collection
- Formally, an iterator is an object of a class which implements the generic interface `Iterator`.

Traversing a collection: iterators

- The iterator can be used to traverse the collection using its `hasNext` and `next` methods
- `next()` gives us the respective next item in the collection
- If `hasNext()` returns `true`, there are further items to traverse. If it returns `false`, we came to the end of the collection.
- Important: Once the traversal has finished, the used iterator became useless! It cannot be reset to the first item in the collection. (However, you could retrieve a new iterator for the same collection and start traversing again)
- Conceptually related to Java 8 data *streams* (which also cannot be "reused")

Iterator Example

```
LinkedList<GeometricShape> myList =
    new LinkedList<GeometricShape>();

...
... // add some items (omitted here)
...
Iterator<GeometricShape> iterator = myList.iterator();
while(iterator.hasNext())
    iterator.next().display();
```

Iterator example: loop

- Alternatively, we can use a *for-each loop* to traverse collections. Note the different syntax compared to ordinary for loops:

```
LinkedList<GeometricShape> myList = new LinkedList<GeometricShape>();
... // add some items (omitted here)
...
```

type of the items in the list

```
for(GeometricShape element: myList) {
    element.display();
    ...
}
```

- With Java 8 streams, you will learn about another approach to traversing data structures which doesn't require loops at all!

End of Topic 3: Java Collections

Topic 4: Java 8 Streams

Java 8 Streams

- One of the most important new features introduced in Java 8 are *streams* (java.util.stream).
- They are strictly speaking not a data structure itself, but a way of sequentially computing data
- Each stream has a *source* where its data comes from, e.g., a collection or a file
- A stream doesn't store itself the data but makes the data accessible to the programmer
- Features of stream over collections and iterators:
    - Many stream operations work lazily: they materialize results only as far as required for computing the end result
    - They don't manipulate their respective sources
    - They can be unbounded, even infinite

Java 8 Streams operations

- Stream *operations* provide computations on the stream elements (e.g. *filtering*, *mapping* or *traversing*)
- *Intermediate* stream operations do so by creating a new stream. Typically, multiple intermediate operations are chained together
- *Terminal* operations return a non-stream result (e.g., individual data items or a collection), or nothing
- Once traversed, existing streams cannot be reduced (but you can create a new stream from an existing stream)

Java 8 Streams operations: Examples

- Filter, e.g.,

```
Stream<Person> personsOver18 = persons.stream()
     .filter(p -> p.getAge() > 18);
```

Lambda expression

- Map, e.g.,

```
Stream<Student> myStream = persons.stream()
          .map(person -> new Student(person));
```

Creates stream from collection "persons"

- ForEach, e.g.,

```
someStream.forEach(p -> p.setLastName("Doe"))   // forEach is a terminal operation
```

- The concrete operation (e.g., the filter condition or the map expression) is typically represented as a lambda expression, but can alternatively also be a method, using a method reference, e.g., `Person::compareAge`
- Therefore, methods such as map or filter need to be higher-order functions (i.e., functions which take other functions as arguments)

Java 8 Streams pipelines

- Chains of operations such as

```
personsList
        .stream()
        .filter(allPilots)
        .mapToInt(p -> p.getAge())
        .sum();
```

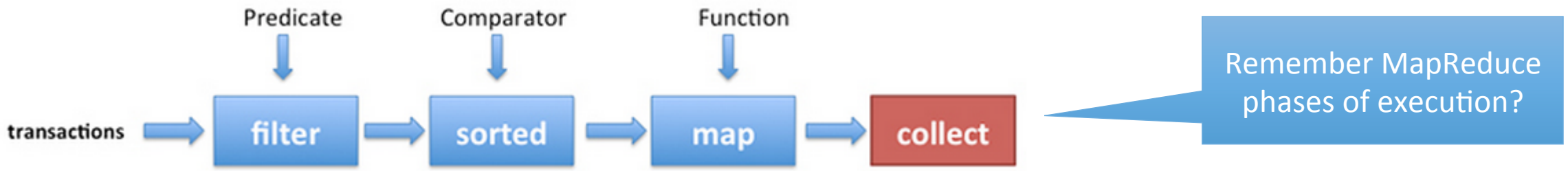
  …are called *pipelines*
- Operations such as sum() or average() are called *aggregate operations* (because they compute a single value from a collection of items) (*)
- Bulk and aggregate operations are also the basic kinds of Big Data Analytics operations such as in the *MapReduce* approach
- Also similar to SQL operations (SELECT … FROM … WHERE …)

(*) Sometimes *all* bulk operations are called "aggregate operations", although that's not entirely correct

Java 8 Streams pipelines: example

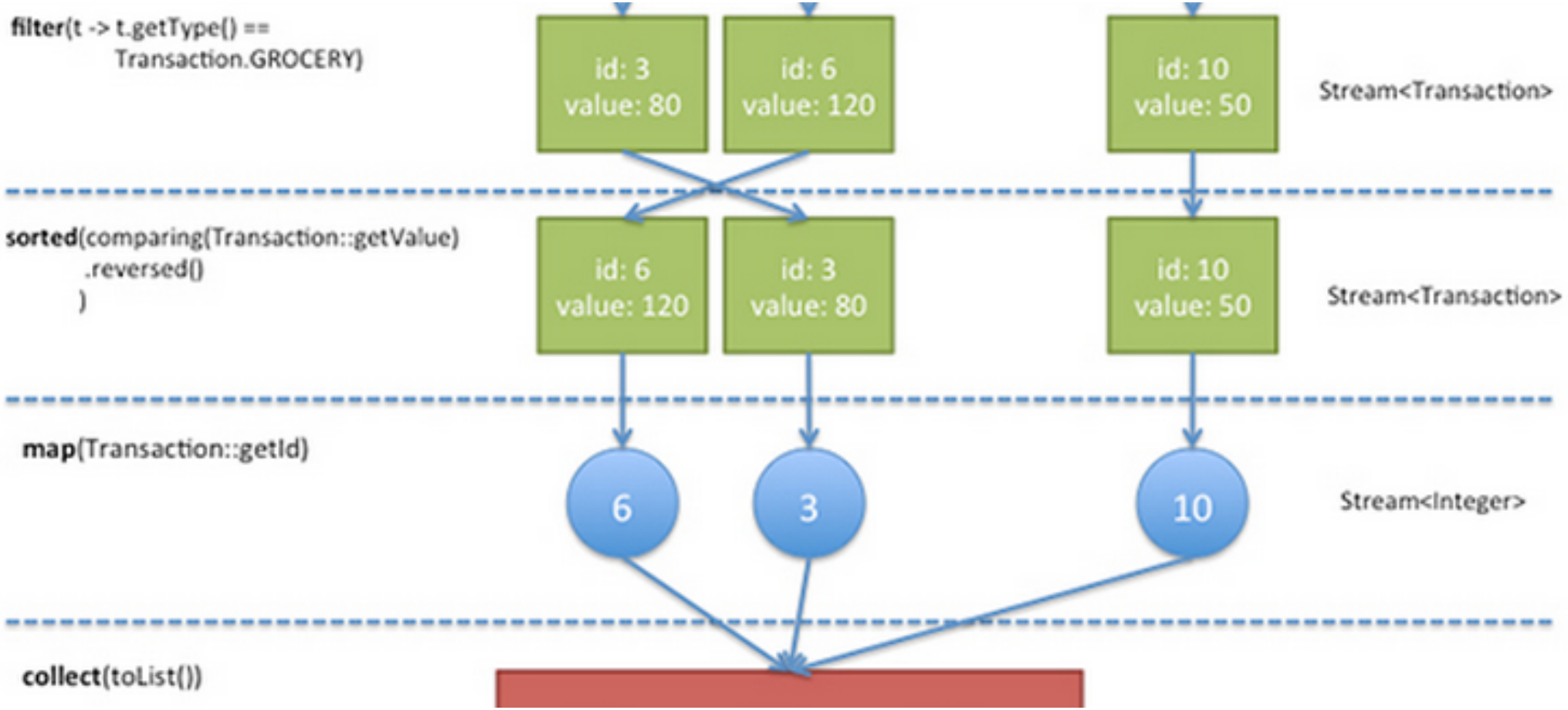
Another, more complex, example for such a pipeline, now using a stream of shopping transactions:

```
List<Integer> transactionsIds = transactions.stream()
// we obtain a stream from collection transactions
        .filter(t -> t.getType() == Transaction.GROCERY)
        .sorted(comparing(Transaction::getValue).reversed())
        .map(Transaction::getId)
        .collect(toList()); // collect creates a list (i.e., a
                            // collection) from the stream
```



Source: Oracle technetwork

Java 8 Streams pipelines: example



Source: Oracle technetwork

Java 8 (parallel) Streams

- Stream operations can *automatically* make use of parallel computing (on multi-core processors), which is much easier than working manually with threads (although parallel streams also make use of threads internally, of course)

```
List<Integer> transactionsIds =
   transactions.parallelStream()
         .filter(t -> t.getType() == Transaction.GROCERY)
         .sorted(comparing(Transaction::getValue).reversed())
         .map(Transaction::getId)
         .collect(toList());
```

Note: Race conditions can still occur! So you should still try to ensure that your data is immutable and that the functions applied on the data don't have side effects (see lecture on multithreading)

- Pipelines of operations can be automatically optimized (so-called *short-circuiting* - merging of operations)
- Streams can even be *infinite* (how that?):

```
Stream<Integer> numbers = Stream.iterate(0, n -> n + 10);   // 0, 10, 20, 30, …
```

Java 8 Streams lazy evaluation

- *Intermediate* stream operations (e.g., map, filter) return a new stream; *terminal* operations such as forEach close the stream
- Because streams are *lazy*, they "materialize" their intermediate operation results only with terminal operations. E.g.,

```
Stream.of("a", "b", "c")
   .filter(x -> {
        System.out.println("filter: " + x);
        return true;
    });
```

doesn't print anything, because there is no terminal operation.

But if you add

```
   .forEach(x -> System.out.println("forEach: " + x));
```

at the end, it prints the items (twice, both in filter and forEach!).

Java 8 Streams

- Similar to iterators, a closed stream cannot be "reused". You cannot iterate over the same stream again using bulk or other operations
- However, we can 1) create a new stream from an existing one (that's what intermediate operations such as map do)
  or 2) convert a stream to an ordinary collection after all stream operations have been done. Ordinary collections can be traversed and modified infinitely often

```
myStream.collect(Collectors.toList());  //converts stream
                                        //to list
```

End of Topic 4:
Java 8 Streams